Kernel frameworks for device drivers



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!

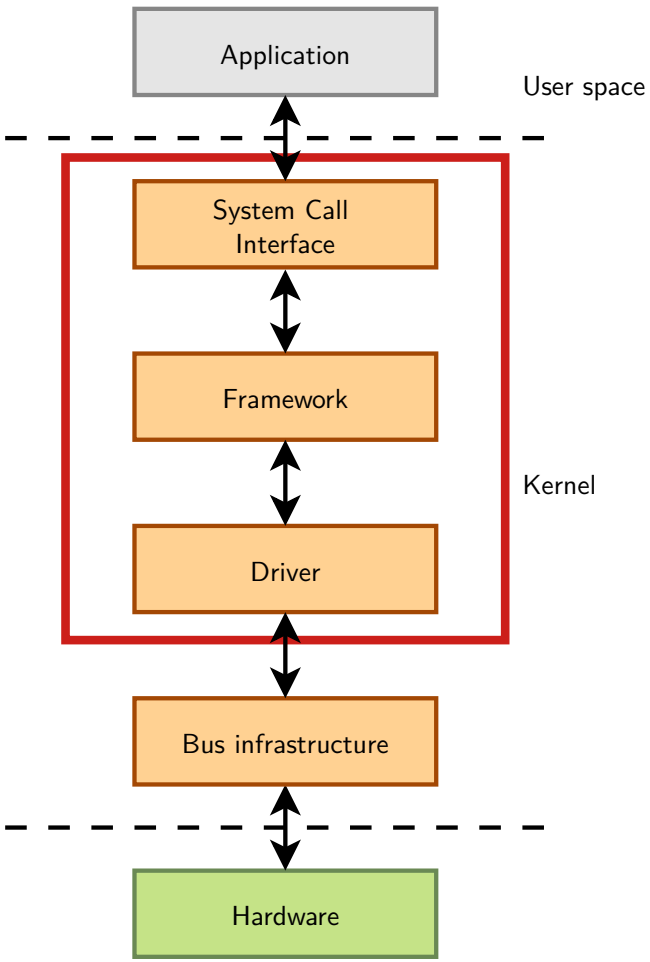


Kernel and Device Drivers

In Linux, a driver is always interfacing with:

- a **framework** that allows the driver to expose the hardware features to user space applications.
- a **bus infrastructure**, part of the device model, to detect/communicate with the hardware.

This section focuses on the *kernel frameworks*, while the *bus infrastructure* was covered earlier in this training.





User space vision of devices



Types of devices

Under Linux, there are essentially four types of devices:

- **Network devices**. They are represented as network interfaces, visible in user space using `ip a`
- **Block devices**. They are used to provide user space applications access to raw storage devices (hard disks, USB keys). They are visible to the applications as *device files* in `/dev`.
- **Character devices**. They are used to provide user space applications access to all other types of devices (input, sound, graphics, serial, etc.). They are also visible to the applications as *device files* in `/dev`.
- **Sysfs devices**. They don't have any of the above user space interfaces, only a representation in sysfs. "Internal" device drivers fall under this (e.g. pinctrl), but also some user-space accessible devices. E.g. gpio (deprecated), IIO (Industrial I/O).

→ Most devices are *character devices*, so we will study these in more details.



Devices: everything is a file

- A very important UNIX design decision was to represent most *system objects* as files
- It allows applications to manipulate all *system objects* with the normal file API (`open`, `read`, `write`, `close`, etc.)
- So, devices had to be represented as files to the applications
- This is done through a special artifact called a **device file**
- It is a special type of file, associating a file name visible to user space applications to a triplet the kernel understands:
  - *type*: `char` or `block`
  - *major*: class of the device
  - *minor*: unique identifier in a class
- All *device files* are by convention stored in the `/dev` directory



Device files examples

Example of device files in a Linux system

```
$ ls -l /dev/ttyS0 /dev/tty1 /dev/sda /dev/sda1 /dev/sda2 /dev/sdc1 /dev/zero
brw-rw---- 1 root disk    8,  0 2011-05-27 08:56 /dev/sda
brw-rw---- 1 root disk    8,  1 2011-05-27 08:56 /dev/sda1
brw-rw---- 1 root disk    8,  2 2011-05-27 08:56 /dev/sda2
brw-rw---- 1 root disk    8, 32 2011-05-27 08:56 /dev/sdc
crw------- 1 root root    4,  1 2011-05-27 08:57 /dev/tty1
crw-rw---- 1 root dialout 4, 64 2011-05-27 08:56 /dev/ttyS0
crw-rw-rw- 1 root root    1,  5 2011-05-27 08:56 /dev/zero
```

Example C code that uses the usual file API to write data to a serial port

```c
int fd;
fd = open("/dev/ttyS0", O_RDWR);
write(fd, "Hello", 5);
close(fd);
```



Creating device files

- Before Linux 2.6.32, on basic Linux systems, the device files had to be created manually using the `mknod` command
  - `mknod /dev/<device> [c|b] major minor`
  - Needed root privileges
  - Coherency between device files and devices handled by the kernel was left to the system developer
- The `devtmpfs` virtual filesystem can be mounted on `/dev` and contains all the devices registered to kernel frameworks. The CONFIG_DEVTMPFS_MOUNT kernel configuration option makes the kernel mount it automatically at boot time, except when booting on an initramfs.
- `devtmpfs` can be supplemented by userspace tools like `udev` or `mdev` to adjust permission/ownership, load kernel modules automatically and create symbolic links to devices.



Character drivers

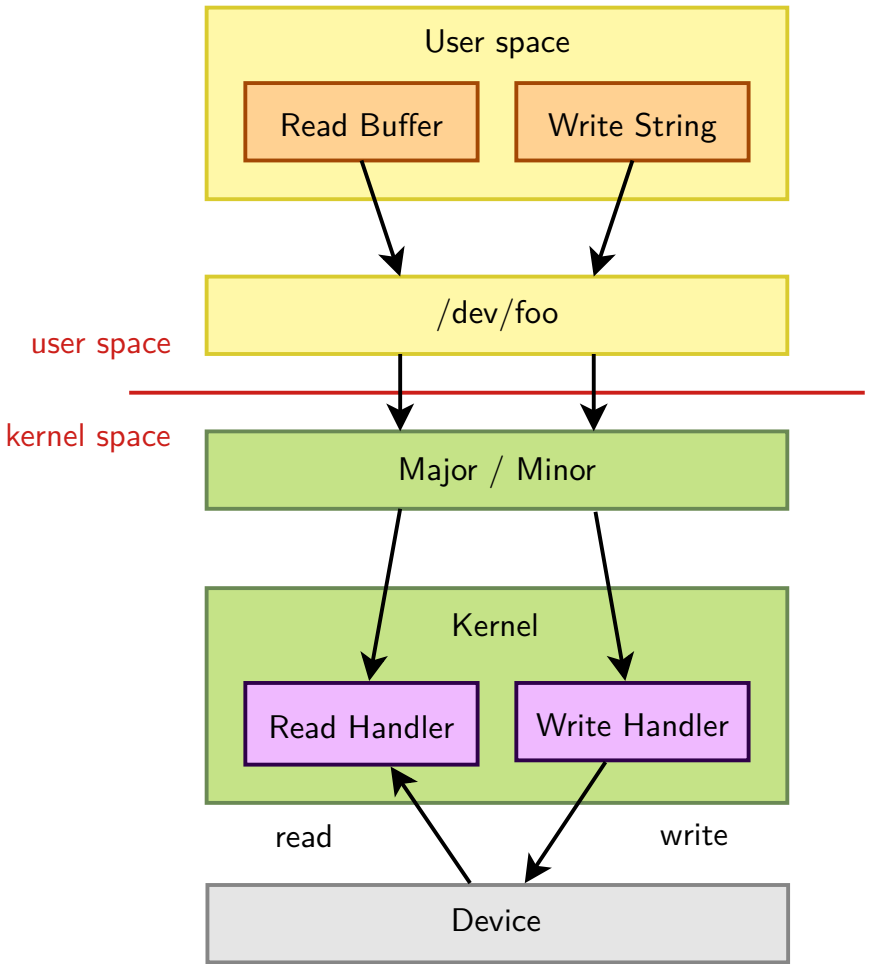


A character driver in the kernel

- From the point of view of an application, a *character device* is essentially a **file**.
- Character device drivers therefore implement **operations** that let applications think the device is a file.
- In order to achieve this, a character driver implements the operations it wants from the struct file_operations structure: `read`, `write`, `ioctl`, etc.
- The Linux filesystem layer will ensure that the driver's operations are called when a user space application makes the corresponding system call.



From user space to the kernel: character devices





File operations

Here are the most important operations for a character driver, from the definition of struct file_operations:

```
struct file_operations {
    struct module *owner;
    ssize_t (*read) (struct file *, char __user *,
        size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *,
        size_t, loff_t *);
    long (*unlocked_ioctl) (struct file *, unsigned int,
        unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*release) (struct inode *, struct file *);
    ...
};
```

Many operations exist, they are all optional.



open() and release()

- `int open(struct inode *i, struct file *f)`
  - Called when user space opens the device file.
  - Only implement this function when you do something special with the device at `open()` time.
  - `struct inode` is a structure that uniquely represents a file in the filesystem (be it a regular file, a directory, a symbolic link, a character or block device)
  - `struct file` is a structure created every time a file is opened. Several file structures can point to the same `inode` structure.
    - Contains information like the current position, the opening mode, etc.
    - Has a `void *private_data` pointer that one can freely use.
    - A pointer to the `file` structure is passed to all other operations
- `int release(struct inode *i, struct file *f)`
  - Called when user space closes the file.
  - Only implement this function when you do something special with the device at `close()` time.



read() and write()

- `ssize_t read(struct file *f, char __user *buf, size_t sz, loff_t *off)`
  - Called when user space uses the `read()` system call on the device.
  - Must read data from the device, write at most `sz` bytes to the user space buffer `buf`, and update the current position in the file `off`. `f` is a pointer to the same file structure that was passed in the `open()` operation
  - Must return the number of bytes read. `0` is usually interpreted by userspace as the end of the file.
  - On UNIX, `read()` operations typically block when there isn't enough data to read from the device
- `ssize_t write(struct file *f, const char __user *buf, size_t sz, loff_t *off)`
  - Called when user space uses the `write()` system call on the device
  - The opposite of `read`, must read at most `sz` bytes from `buf`, write it to the device, update `off` and return the number of bytes written.



Exchanging data with user space 1/3

- Kernel code isn't allowed to directly access user space memory, using memcpy() or direct pointer dereferencing
  - User pointer dereferencing is disabled by default to make it harder to exploit vulnerabilities.
  - If the address passed by the application was invalid, the kernel could segfault.
  - **Never** trust user space. A malicious application could pass a kernel address which you could overwrite with device data (`read` case), or which you could dump to the device (`write` case).
  - Doing so does not work on some architectures anyway.
- To keep the kernel code portable, secure, and have proper error handling, your driver must use special kernel functions to exchange data with user space.

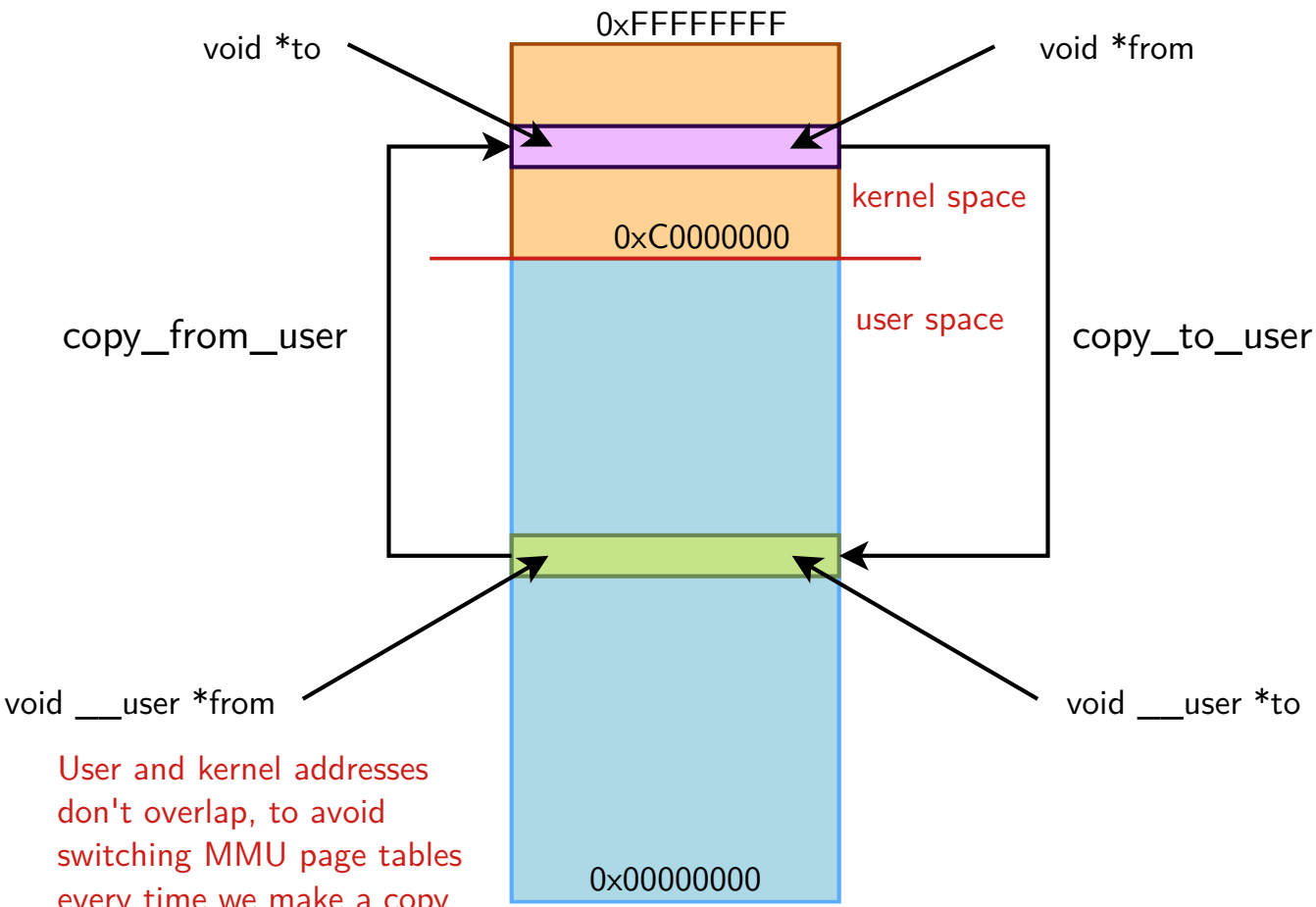


Exchanging data with user space 2/3

- A single value
  - `get_user(v, p);`
    - The kernel variable `v` gets the value pointed by the user space pointer `p`
  - `put_user(v, p);`
    - The value pointed by the user space pointer `p` is set to the contents of the kernel variable `v`.
- A buffer
  - ```
    unsigned long copy_to_user(void __user *to, const void *from,
                               unsigned long n);
    ```
  - ```
    unsigned long copy_from_user(void *to, const void __user *from,
                                 unsigned long n);
    ```
- The return value must be checked. Zero on success, non-zero on failure. If non-zero, the convention is to return -EFAULT.



Exchanging data with user space 3/3





Zero copy access to user memory

- Having to copy data to or from an intermediate kernel buffer can become expensive when the amount of data to transfer is large (video).
- *Zero copy* options are possible:
  - `mmap()` system call to allow user space to directly access memory mapped I/O space. See our `mmap()` chapter.
  - get_user_pages() and related functions to get a mapping to user pages without having to copy them.



unlocked_ioctl()

- `long unlocked_ioctl(struct file *f, unsigned int cmd, unsigned long arg)`
  - Associated to the `ioctl()` system call.
  - Called unlocked because it didn't hold the Big Kernel Lock (gone now).
  - Allows to extend the driver capabilities beyond the limited read/write API.
  - For example: changing the speed of a serial port, setting video output format, querying a device serial number... Used extensively in the V4L2 (video) and ALSA (sound) driver frameworks.
  - `cmd` is a number identifying the operation to perform.
    See driver-api/ioctl for the recommended way of choosing `cmd` numbers.
  - `arg` is the optional argument passed as third argument of the `ioctl()` system call. Can be an integer, an address, etc.
  - The semantic of `cmd` and `arg` is driver-specific.



ioctl() example: kernel side

```
#include <linux/phantom.h>

static long phantom_ioctl(struct file *file, unsigned int cmd,
                          unsigned long arg)
{
        struct phm_reg r;
        void __user *argp = (void __user *)arg;

        switch (cmd) {
        case PHN_SET_REG:
                if (copy_from_user(&r, argp, sizeof(r)))
                        return -EFAULT;
                /* Do something */
                break;
        ...
        case PHN_GET_REG:
                if (copy_to_user(argp, &r, sizeof(r)))
                        return -EFAULT;
                /* Do something */
                break;
        ...
        default:
                return -ENOTTY;
        }

        return 0;
}
```

Selected excerpt from drivers/misc/phantom.c



Ioctl() Example: Application Side

```c
#include <linux/phantom.h>

int main(void)
{
        int fd, ret;
        struct phm_reg reg;

        fd = open("/dev/phantom");
        assert(fd > 0);

        reg.field1 = 42;
        reg.field2 = 67;

        ret = ioctl(fd, PHN_SET_REG, &reg);
        assert(ret == 0);

        return 0;
}
```



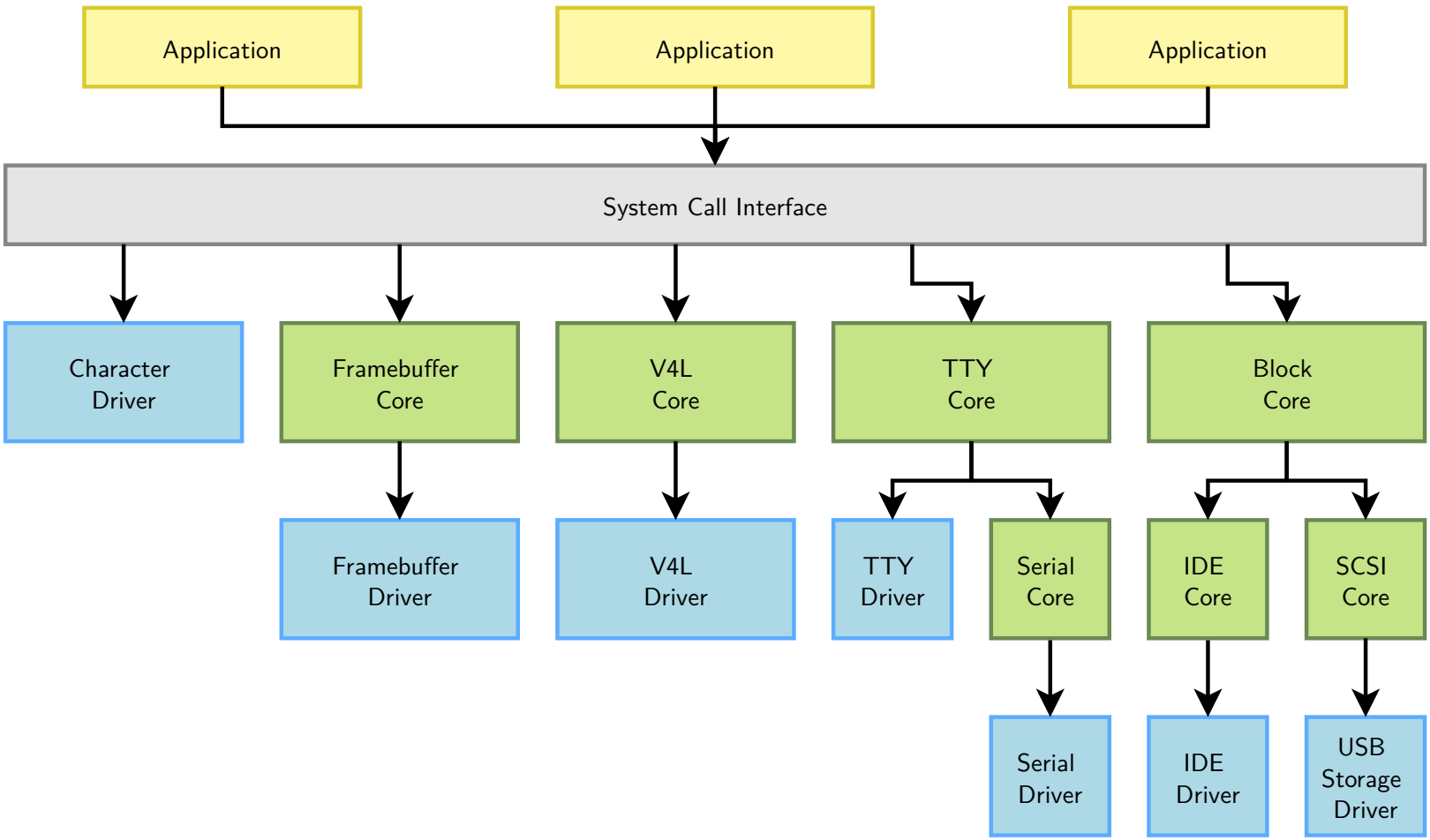
The concept of kernel frameworks



Beyond character drivers: kernel frameworks

- Many device drivers are not implemented directly as character drivers
- They are implemented under a *framework*, specific to a given device type (framebuffer, V4L, serial, etc.)
  - The driver plugs into a framework API rather than into the generic struct file_operations
  - The framework implements once struct file_operations to expose character devices to userspace
  - That implementation is shared across all framework devices: it minimizes driver boilerplate and it provides a coherent userspace interface whatever driver is being used

Example: Some Kernel Frameworks





Example: the input subsystem

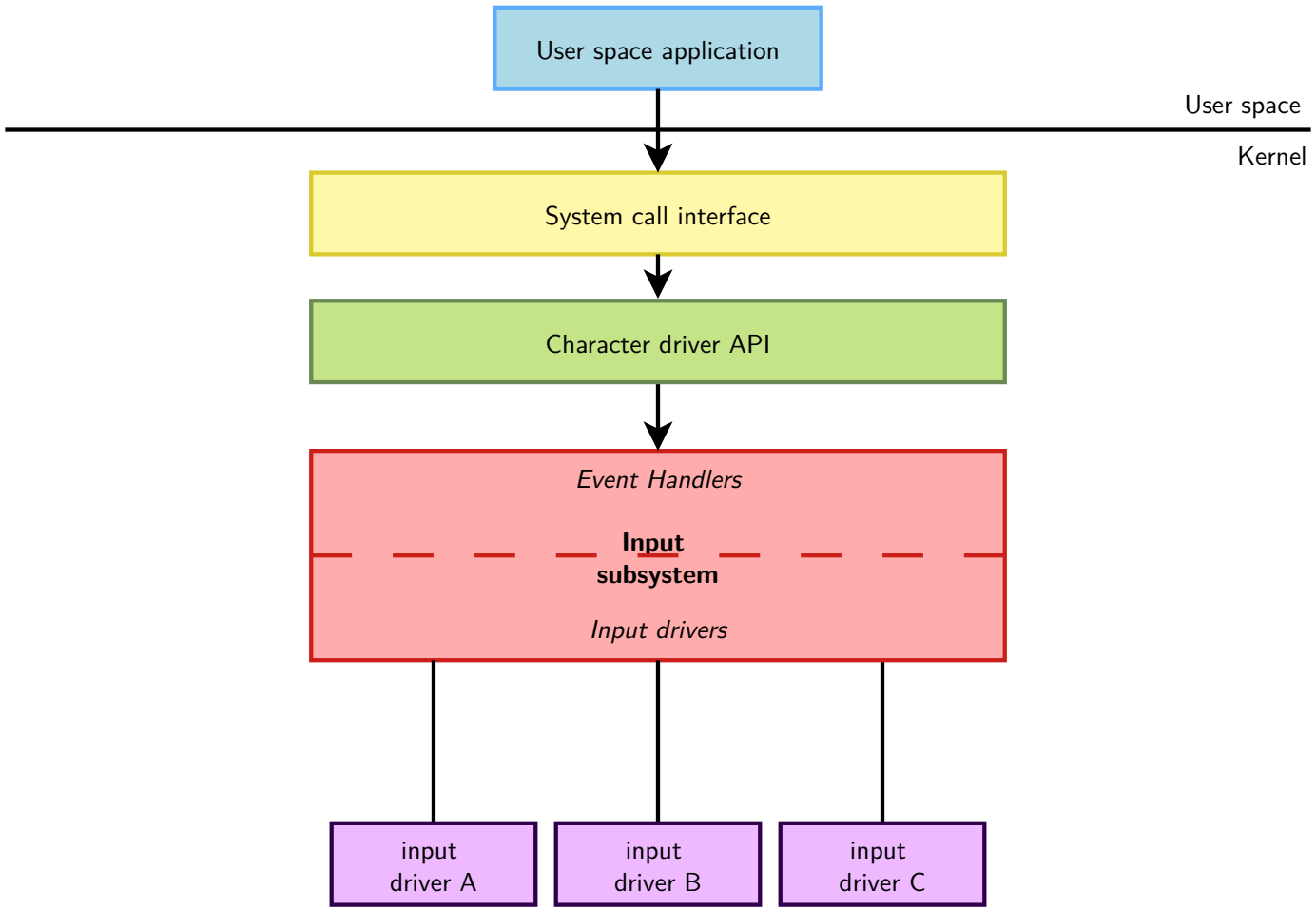


What is the input subsystem?

- The input subsystem takes care of all the input events coming from the human user.
- Initially written to support the USB *HID* (Human Interface Device) devices, it quickly grew up to handle all kinds of inputs (using USB or not): keyboards, mice, joysticks, touchscreens, etc.
- The input subsystem is split in two parts:
  - **Device drivers**: they talk to the hardware (for example via USB), and provide events (keystrokes, mouse movements, touchscreen coordinates) to the input core
  - **Event handlers**: they get events from drivers and pass them where needed via various interfaces (most of the time through `evdev`)
- In user space it is usually used by the graphic stack such as *X.Org*, *Wayland* or *Android's InputManager*.



Input subsystem diagram





Input subsystem overview

- Kernel option CONFIG_INPUT
  - `menuconfig INPUT`
    - `tristate "Generic input layer (needed for keyboard, mouse, ...)"`
- Implemented in drivers/input/
  - input.c, input-poller.c, evdev.c...
- Defines the user/kernel API
  - include/uapi/linux/input.h
- Defines the set of operations an input driver must implement and helper functions for the drivers
  - struct input_dev for the device driver part
  - struct input_handler for the event handler part
  - include/linux/input.h



Input subsystem API 1/3

An *input device* is described by a very long struct input_dev structure, an excerpt is:

```
struct input_dev {
    const char *name;
    [...]
    struct input_id id;
    [...]
    unsigned long evbit[BITS_TO_LONGS(EV_CNT)];
    unsigned long keybit[BITS_TO_LONGS(KEY_CNT)];
    [...]
    int (*getkeycode)(struct input_dev *dev,
                      struct input_keymap_entry *ke);
    [...]
    int (*open)(struct input_dev *dev);
    [...]
    int (*event)(struct input_dev *dev, unsigned int type,
                 unsigned int code, int value);
    [...]
};
```

Before being used, this structure must be allocated and initialized, typically with:

```
struct input_dev *devm_input_allocate_device(struct device *dev);
```



Input subsystem API 2/3

- Depending on the type of events that will be generated, the input bit fields `evbit` and `keybit` must be configured: For example, for a button we only generate EV_KEY type events, and from these only BTN_0 events code:

```
set_bit(EV_KEY, myinput_dev.evbit);
set_bit(BTN_0, myinput_dev.keybit);
```

- Once the *input device* is allocated and filled, the function to register it is:
  `int input_register_device(struct input_dev *);`



Input subsystem API 3/3

The events are sent by the driver to the event handler using

```
void input_event(struct input_dev *dev, unsigned int type, unsigned int code, int value)
```

- The event types are documented in input/event-codes
- An event is composed by one or several input data changes (packet of input data changes) such as the button state, the relative or absolute position along an axis, etc..
- The input subsystem provides other wrappers such as:
  - input_report_key()
  - input_report_abs()

After submitting potentially multiple events, the *input* core must be notified by calling:

```
void input_sync(struct input_dev *dev)
```



Example from drivers/hid/usbhid/usbmouse.c

```
static void usb_mouse_irq(struct urb *urb)
{
        struct usb_mouse *mouse = urb->context;
        signed char *data = mouse->data;
        struct input_dev *dev = mouse->dev;
        ...

        input_report_key(dev, BTN_LEFT,   data[0] & 0x01);
        input_report_key(dev, BTN_RIGHT,  data[0] & 0x02);
        input_report_key(dev, BTN_MIDDLE, data[0] & 0x04);
        input_report_key(dev, BTN_SIDE,   data[0] & 0x08);
        input_report_key(dev, BTN_EXTRA,  data[0] & 0x10);

        input_report_rel(dev, REL_X,     data[1]);
        input_report_rel(dev, REL_Y,     data[2]);
        input_report_rel(dev, REL_WHEEL, data[3]);

        input_sync(dev);
        ...
}
```



Polling input devices

- The input subsystem provides an API to support simple input devices that *do not raise interrupts* but have to be *periodically scanned or polled* to detect changes in their state.
- Setting up polling is done using input_setup_polling():

```
int input_setup_polling(struct input_dev *dev, void (*poll_fn)(struct input_dev *dev));
```

- `poll_fn` is the function that will be called periodically.
- The polling interval can be set using input_set_poll_interval() or input_set_min_poll_interval() and input_set_max_poll_interval()



evdev user space interface

- The main user space interface to *input devices* is the **event interface**
- Each *input device* is represented as a `/dev/input/event<X>` character device
- A user space application can use blocking and non-blocking reads, but also `select()` (to get notified of events) after opening this device.
- Each read will return struct input_event structures of the following format:

```
struct input_event {
        struct timeval time;
        unsigned short type;
        unsigned short code;
        unsigned int value;
};
```

- A very useful application for *input device* testing is `evtest`, from https://cgit.freedesktop.org/evtest/



Device-managed allocations



Device managed allocations

- The `probe()` function is typically responsible for allocating a significant number of resources: memory, mapping I/O registers, registering interrupt handlers, etc.
- These resource allocations have to be properly freed:
  - In the `probe()` function, in case of failure
  - In the `remove()` function
- This required a lot of failure handling code that was rarely tested
- To solve this problem, *device managed* allocations have been introduced.
- The idea is to associate resource allocation with the `struct device`, and automatically release those resources
  - When the device disappears
  - When the device is unbound from the driver
- Functions prefixed by `devm_`
- See driver-api/driver-model/devres for details



Device managed allocations: memory allocation example

- Normally done with `kmalloc(size_t, gfp_t)`, released with `kfree(void *)`
- Device managed with `devm_kmalloc(struct device *, size_t, gfp_t)`

Without devm functions

```
int foo_probe(struct platform_device *pdev)
{
        struct foo_t *foo = kmalloc(sizeof(struct foo_t),
                                    GFP_KERNEL);
        /* Register to framework, store
         * reference to framework structure in foo */
        ...
        if (failure) {
                kfree(foo);
                return -EBUSY;
        }
        platform_set_drvdata(pdev, foo);
        return 0;
}

void foo_remove(struct platform_device *pdev)
{
        struct foo_t *foo = platform_get_drvdata(pdev);
        /* Retrieve framework structure from foo
           and unregister it */
        ...
        kfree(foo);
}
```

With devm functions

```
int foo_probe(struct platform_device *pdev)
{
        struct foo_t *foo = devm_kmalloc(&pdev->dev,
                                         sizeof(struct foo_t),
                                         GFP_KERNEL);
        /* Register to framework, store
         * reference to framework structure in foo */
        ...
        if (failure)
                return -EBUSY;
        platform_set_drvdata(pdev, foo);
        return 0;
}

void foo_remove(struct platform_device *pdev)
{
        struct foo_t *foo = platform_get_drvdata(pdev);
        /* Retrieve framework structure from foo
           and unregister it */
        ...
        /* foo automatically freed */
}
```



Device managed allocations caveats

- Cleanup is done when the `struct device` is cleaned up. There is no reference counting or anything like that.
- Don't use if the allocated memory is used outside of the device node. E.g. if the userspace device file is still open after remove.
- Be very careful when there are circular references.
- "Why is `devm_kzalloc()` harmful and what can we do about it", Laurent Pinchart, LPC 2022



Driver data structures and links



Driver data layout Three main data structures:

- Bus-specific device structure (struct i2c_client, struct usb_dev, etc)
  - It *always embeds* a struct device
  - A pointer to it is passed to the `probe()` function
- Framework-specific device struct (struct input_dev, struct rtc_device, etc)
  - It *might embed* a struct device if the framework wants a sysfs device
  - Careful! We might have one bus struct device and one framework struct device for the same device!
- Driver private data
  - The structure is driver specific, with space for all device state information
  - It stores references to both the bus and framework devices



Driver data allocation strategies

Private data embeds the framework device therefore a single allocation allocates both the framework device and the private data.

```
struct imx_port {
        struct uart_port port;
        struct timer_list timer;
        unsigned int old_status;
        int txirq, rxirq, rtsirq;
        [...]
};

sport = devm_kzalloc(&pdev->dev,
                     sizeof(*sport),
                     GFP_KERNEL);
if (!sport)
        return -ENOMEM;
```

The framework exposes an helper to allocate the framework device, with space at the end to put the private data.

```
struct da311_data *data;
struct iio_dev *idev;

idev = devm_iio_device_alloc(
                     &client->dev,
                     sizeof(*data));
if (!idev)
        return -ENOMEM;

data = iio_priv(idev);
data->client = client;
```

The framework device and private data are allocated separately.

```
struct rtc_device *rtc;
struct ds1305 *ds1305;

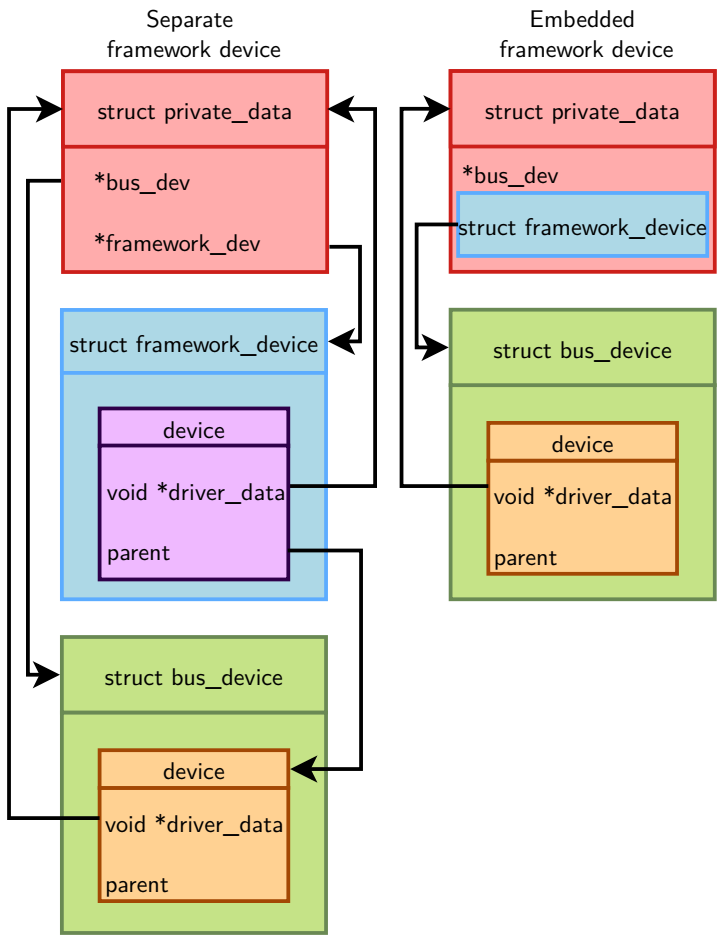
ds1305 = devm_kzalloc(&spi->dev,
                      sizeof(*ds1305),
                      GFP_KERNEL);
if (!ds1305)
        return -ENOMEM;

rtc = devm_rtc_allocate_device(...);
if (IS_ERR(rtc))
        return PTR_ERR(rtc);
```



Links between data structures

- Inside bus callbacks, we get passed our bus device
  - Inside struct device, a pointer-sized field `dev->driver_data` is reserved for driver usage
  - Use dev_set_drvdata() at `probe()` time to put a reference to our private data
  - From bus callbacks, we can retrieve our private data using dev_get_drvdata()
- Inside framework callbacks, we get passed our framework device
  - If our framework device is embedded in our private data, we use container_of() that works using compiler provided `offsetof()`
  - Otherwise, we use the framework device `dev->driver_data` and retrieve our private data reference



Practical lab - Expose the Nunchuk to user space



- Extend the Nunchuk driver to expose the Nunchuk features to user space applications, as an *input* device.
- Test the operation of the Nunchuk using `evtest`